# LWTV

## Android malware audit
## and attacker-path reconstruction

com.yunyou.control.dev
Version 274.533.185 | 2 November 2026

**VERDICT: HIGH-CONFIDENCE MALWARE**
Concealed Android remote control, credential capture and phishing capability.

**Known initial control origin**
www.capebej.com / 213.111.157.6
DNS observation: 2 November 2026, 06:01:27 UTC

**What this report establishes**
The hidden program ran in an isolated emulator. Source and bytecode link credential collectors to local raw-value storage and an authenticated upload route.

**What remains unknown**
Outbound blocking prevented live enrollment, remote commands and exfiltration. Final backend storage, onward recipients and the alleged financial transaction were not established.

Prepared from the supplied APK, authenticated payload unpacking, targeted reverse engineering and an offline Android sandbox. The companion ZIP preserves indicators and selected supporting evidence.

## Contents


Executive assessment 3

1. Sample identity and scope 4

2. Attack path reconstructed from the client 5

3. Permissions, entry points and persistence 7

4. Packing and anti-analysis details 8

5. How credentials are captured 9

6. The credential cache and exact upload path 11

7. Other data collected or exposed 12

8. Remote takeover and concealment 14

9. Network destinations and route inventory 15

10. Payloads and downloads 18

11. Sandbox procedure and observations 19

12. Confidence, alternatives and unresolved questions 21

13. Actions for an affected phone and financial-loss investigation 22

14. Evidence package and cleanup note 23

# Executive assessment

**Assessment date:** 2 November 2026. **Sample:** LWTV_com.yunyou.control.dev_v274.533.185.apk. **Verdict:** High-confidence malicious Android remote-control and credential-capture application. Treat this APK as unsafe.

This report combines inspection of the original APK, authenticated unpacking of two hidden DEX payloads, source and bytecode analysis, and execution in a disposable Android emulator with outbound traffic blocked. It traces the implemented paths from installation to collection and the last network destinations recoverable from the client. A successful theft, live operator session, or the reported financial loss was not reproduced or independently verified.

The requester reports that they did not install the APK. The account of a friend's friend's phone being hijacked and money lost is contextual information, not forensic evidence from that phone.

## Principal conclusions

- The APK hides its main code in an AES-GCM encrypted asset. Both extracted DEX files match the DEX files recovered from the running sample's memory.
- Accessibility is the central takeover mechanism. Code reads interface events, performs gestures and navigation, collects screen information, manipulates overlays, and attempts to capture credentials. No root exploit is needed for these implemented paths once the relevant access is granted.
- A dedicated credential queue stores raw values, including typed text, app passwords, PIN/pattern-related captures and phishing form submissions. Its upload endpoint is **https://www.capebej.com/m9eu/00/0v7**, unless an accepted endpoint update changes the base origin.
- The configured API/control host **www.capebej.com** resolved to **213.111.157.6** at **2026-11-02 06:01:27 UTC**. This is a point-in-time DNS result, not proof of the server used during the alleged victim incident or the final storage location behind that server.
- The code supports additional server-supplied HTTPS origins, an encrypted MQTT broker, and overlay page URLs. Their live values were not obtained. Backend databases, onward transfers, operators and beneficiaries cannot be established from this APK alone.
- The sandbox confirmed code loading and Accessibility activation. Enrollment failed because outbound traffic was blocked. No credential upload, remote command execution, or financial transaction was observed.

## Reading the evidence

**Static evidence** means a code path or configuration was recovered from the sample. **Runtime evidence** means the behavior was observed in the emulator. **Inference** explains plausible consequences of those capabilities. **Unknown** marks information the APK and isolated test cannot establish. These categories are kept separate throughout the report.

Source references refer to the recovered, normalized Java listings, with original decompiler line positions retained. `deviceconnection-full.java` is a second, targeted decompilation and has a separate line numbering scheme. Decompiled Java is an analysis aid: some methods are incomplete. Selected DEX disassemblies were also checked where Java reconstruction failed. The companion evidence document preserves important excerpts so the report remains useful after the requested local cleanup.

# 1. Sample identity and scope

| Attribute | Value |
|---|---|
| Original filename | LWTV_com.yunyou.control.dev_v274.533.185.apk |
| Original size | 6,158,481 bytes |
| SHA-256 | 056a8db666e4f154f9f6d0b7a00aa0703d2990ee2b1a085b7339a49bdef90495 |
| Package / launcher label | com.yunyou.control.dev / LWTV |
| Manifest version | 274.533.185, versionCode 2053099 |
| Internal BuildConfig version | 0.2.0, versionCode 68 |
| SDK settings | minSdk 28, targetSdk 34, compileSdk 36 |
| APK signature | v3 verifies; v1 and v2 absent; one signer |
| Signing certificate SHA-256 | a40da80a59d170caa950cf15c18c454d47a39b26989d8b640ecd745ba71bf5dc |

The manifest/internal version discrepancy is consistent with packaging or relabeling; it does not identify who performed it. The certificate subject says Android, with an android@android.com email and Mountain View address. Those are self-asserted certificate fields, not evidence of Google authorship or trust.

The original path was `C:\Users\Amarnath\Downloads\AyuGram Desktop\LWTV_com.yunyou.control.dev_v274.533.185.apk`. Being stored in this directory does not establish that Telegram or AyuGram produced the file. The original distribution message, sender, download URL, and victim handset were unavailable.

Scope covers the supplied APK and its embedded code. It does not include access to the suspected controller's infrastructure, a victim device image, banking records, or a live server account. No sample was uploaded to a public malware-analysis service. Host DNS queries were performed; the sample was prevented from contacting external hosts during execution.

# 2. Attack path reconstructed from the client

## Stage A - Delivery and installation

The user receives and sideloads an APK labelled LWTV. The delivery pretext and actual victim clicks are unknown. Opening the recovered HomeActivity can display an embedded Accessibility guide and a WebView cover URL. `HomeActivity.java:243,265-294,544` shows the guide, configured cover, and the action opening Android Accessibility settings.

The cover is configured as **https://famelack.com/**. It is evidence of a presentation destination, not a proven credential-upload endpoint. The cover page could not load in the blocked emulator, so its appearance and contents were not established.

## Stage B - Hidden code activation

The manifest Application class is `com.wipjt.payvpkyvn.qczs.wfqq.QBCf3JUtW31HoXBzXWrLzqj9ufczaS`. It decrypts the bundled payload and loads `com.yunyou.control.AgentApplication`. This places the substantial control and credential-capture implementation behind a small loader and encrypted asset.

Runtime memory extraction recovered both hidden DEX files with the same hashes as offline decryption. Thus the unpacked program is not merely an unused embedded resource: the loader actually mapped it during execution.

## Stage C - Obtain Accessibility and related access

The important escalation is a user enabling the service in Android settings. `AssistanceAccessibilityService.onServiceConnected` initializes the runtime, registers event processing and schedules ongoing work. Its event callback feeds `GPaySniffer` and `NativeController`.

This is abuse of powerful Android facilities after access is granted, not evidence that simply receiving an APK silently roots a phone. Whether the reported victim granted access, bypassed Android's restricted-settings warning, or used a particular OEM setup flow is unknown.

## Stage D - Enroll the installation with the controller

The device consults an endpoint manifest, obtains a challenge, submits a signed enrollment containing an installation identity and embedded enrollment token, then authenticates. Server responses establish the session and control configuration. The bundled enrollment policy is `ACCOUNT_BOUND_V1`; a grant identifier is present. The usable enrollment token is intentionally omitted from the shared report.

## Stage E - Remote observation and control

The client opens an authenticated WebSocket and can operate an encrypted MQTT control transport with server-supplied credentials. The controller can configure observation, request actions, present deceptive overlays, and initiate credential capture. Leases, command sequences and generation checks occur in the generic command pipeline; a known endpoint alone is not enough to issue valid commands.

## Stage F - Collect information and credentials

Accessibility events and interface inspection can expose text and the current app context. Separate collectors attempt PIN, pattern and password extraction. Overlay WebViews send their submitted form data into the same credential cache. Screenshots, interface trees, an initial SMS snapshot and app inventory have separate upload paths.

### Stage G - Upload to the selected API origin

Credential entries are wrapped in a `captures` array and sent to `/m9eu/00/0v7`; the configured origin is www.capebej.com. Other data classes use the routes listed in section 9. TLS encrypts the network leg; it does not prevent the receiving service from reading the submitted values. Local raw-value storage and upload are directly supported by code.

### Stage H - Potential financial abuse

An operator who gains usable credentials and UI control could view an account, navigate a payment app, enter information and potentially authorize a transfer. The screen-covering and settings-blocking code could hinder the owner's response. This is a plausible mechanism for financial loss, not a reconstruction of the reported transaction. Actual success depends on the bank/app, secure-screen behavior, device version, required factors and credentials acquired. No beneficiary account, amount, or completed transfer was recovered.

# 3. Permissions, entry points and persistence

The parsed manifest contains Internet and network-state access, audio-setting modification, foreground-service permissions (including special-use and data-sync), wake lock, exact alarm, boot reception, notifications, battery-optimization exemption request, READ_SMS, RECEIVE_SMS, USE_FINGERPRINT, USE_BIOMETRIC and WRITE_SETTINGS.

Important components include HomeActivity, CredentialCaptureActivity, PermissionActivity, AssistanceAccessibilityService, ConnectionForegroundService, FcmWakeService, LifecycleReceiver, SmsReceiver, KeepaliveRecoveryAlarm and KeepaliveRecoveryJob. The application disables backup in its manifest. The foreground connection service is not stopped simply when its task is removed.

The service requires `BIND_ACCESSIBILITY_SERVICE`, protecting who may bind it; this declaration does not mean the app can grant itself access. In the emulator, Android reported the LWTV service bound, all event types, generic feedback, and capability value 169 (0xA9): retrieve window content, filter key events, perform gestures and take screenshots. Declared capabilities and service binding do not prove every collector succeeds against every screen.

Persistence-related evidence includes boot/unlock/package-replacement receiver actions, foreground work, alarms/jobs, battery/OEM setup logic, FCM wake handling and a service watchdog. These mechanisms are conditional on Android policy and the app's stored enabled/stopped state. Surviving a reboot, defeating force-stop or remaining active on every OEM was not dynamically established.

`AppProtector.java` recognizes settings and package-management interfaces associated with uninstall, force-stop, disabling access and clearing data. It includes BACK/HOME actions to divert the user away from blocked screens (`:370-380,1330,1413+`). This is strong evidence of removal-interference capability. In this sandbox, the logs explicitly said protection was disabled during onboarding; uninstall blocking was not observed.

No device-admin receiver was established in the parsed manifest. The takeover paths described here are centered on Accessibility. Biometric permissions do not give the app fingerprint images or biometric templates.

# 4. Packing and anti-analysis details

The uncompressed binary manifest is 56,179,860 bytes despite the APK being about 6.2 MB. It includes extensive junk attributes and custom permission-like strings. Many duplicate `name` attributes lack the Android namespace. Those were separated from genuine Android attributes during parsing; treating all of them as real component names or permissions would produce false findings. Standard resource/manifest tooling encountered errors.

The loader DEX also contains misleading trailing fill-array-data blocks with invalid element-width values that obstructed normal decompilation. A copy was repaired for analysis by replacing the identified fake trailing blocks with NOPs and updating DEX integrity fields. The original APK was not rewritten or re-signed for the runtime test.

The encrypted asset is **assets/VOUALmvLXIYX**. Its relevant format is: skip 54 bytes; read a big-endian 32-bit DEX count of 2; for each item read a 12-byte nonce, a big-endian ciphertext length and the ciphertext. AES-256-GCM uses a 128-bit tag. Authenticated plaintext is gzip-compressed DEX data.

The loader derives its AES key by XORing these two 32-byte constants:

```
ae6fc4767f061a61d9d125098b95c4fd1b77b0ecba204fd68544ed53a88830d3
892b2ce5499cda8305aeb1cd8442f1ec42279302840def82e90cf4da7b7d1c65
```

AAD consists of the hex-decoded bytes of build identifier `e3be74b947cf6150bf1e43c40aacec32`, followed by big-endian integer 1 and the DEX index. Decryption successfully authenticated both items. There are 35,923 remaining asset bytes outside the two parsed items; they were not needed by this loading path and were not established as another executable payload.

The API 29+ branch uses InMemoryDexClassLoader; the API 28 branch writes read-only JAR containers. The recovered payloads are:

| Payload | Size | SHA-256 |
|---|---|---|
| classes1.dex | 9,959,240 | cf6153e8e4b78cff0526df2d4580cbb48f816a460ec719808303c4e59927eb12 |
| classes2.dex | 1,743,676 | 146718378e77c21aea479223be474d4fb6a27799052fbfca6e8f81c9d9f0eb8f |

Both hashes were independently reproduced from the running process's anonymous DEX mappings. JADX recovered approximately 450 application Java files. Its full run reported 94 errors; targeted DeviceConnection decompilation still reported seven. This limits claims of exhaustive source recovery. The principal capture, upload and control findings are supported by readable methods, associated implementations and selected original DEX instructions.

# 5. How credentials are captured

## 5.1 Event-driven text and password collection

`AssistanceAccessibilityService.java:146-156` forwards Accessibility events to NativeController. The latter processes text changes, clicks, descriptions and lockscreen context. `SequentialTokenBuffer.java:177` submits a capture labelled `keylog`. `MaskedFieldExtractor.java:136,186,237,359,413` contains PIN, password and app-password submissions. `TransactionFieldCapture.java:506-510` submits field-related app-password captures. These labels are backed by data submission code, not only strings found in a binary.

Android may redact password nodes or prevent a service from seeing particular sensitive fields. Capturing a visible digit, reconstructing text-change events and reading a password field are distinct mechanisms; none establishes universal access to every secret. The sandbox did not confirm a successful raw password or UPI PIN capture.

## 5.2 Touch, PIN and pattern paths

`GridSampleTracker.java:401-405` dispatches a recovered pattern to a capture session or cache. `TouchOverlayCapture.java:386-401` records a resolved PIN when available, or a `touch_pin` record with touch points. CredentialCaptureSession also contains a 3-by-3 pattern mapping and digit/character buffers.

The code requests motion-event sources on Android 34+. Full callback connectivity and successful operation on a real secure lockscreen were not demonstrated. Touch coordinates should therefore be reported as collection capability, not as proof that the app can always recover a pattern from Android's protected credential UI.

## 5.3 Credential confirmation prompt

The `CREDENTIAL_CAPTURE` message invokes CredentialCaptureSession.start with controller-supplied title and description (`deviceconnection-full.java:4001-4011,4350-4351`). Defaults include **Identity Verification** and **For your security, please verify your identity**. Starting requires Accessibility and a secure keyguard.

CredentialCaptureActivity invokes Android BiometricPrompt with DEVICE_CREDENTIAL allowed. Android's credential confirmation returns authentication success/failure, not the user's PIN or password. This sample separately attempts to observe input and associates captured content with that result. `CredentialCaptureSession.java:314-318` marks a submitted capture sensitive and verified and requests a flush after authentication succeeds. Foreground/cancel handling can reopen the prompt after roughly 800 ms.

This combination can support social engineering that asks an owner to unlock or verify while capture logic is active. It is not evidence of biometric-template extraction or a cryptographic defeat of Android's credential storage.

## 5.4 WebView phishing overlays

`INJECT_OVERLAY` supplies an action, URL, target package and fullscreen/reset parameters (`deviceconnection-full.java:4170+`). `INJECT_RULES` updates foreground-app matching rules. InjectOverlayManager enables JavaScript, exposes an object named **FKB**, and loads the supplied URL (`:507,522,553`).

The JavaScript bridge is the decisive source-to-upload link:

- `FKB.setFormData(data)` submits raw data as capture type `inject`, with the target package, then flushes (`InjectOverlayManager.java:602-607`).
- `FKB.setLockData(data)` submits raw data as `lockscreen` and flushes (`:684-687`).

The method called `trustedTemplateUrl` only requires an HTTP(S) scheme, nonempty host and a path beginning **/static/dy/templates/** (`:714-719`). It does not pin the host. Thus an accepted command can nominate a separate phishing-page origin. The app manifest disallows cleartext traffic, so acceptance by this string check does not establish that an HTTP page will load under the effective Android network policy. HTTPS is supported.

An overlay page can send its captured form data through FKB into the app's API uploader. The page could also contain its own network behavior, but the live template HTML was unavailable and no additional form destination was established.

### 5.5 Native imitation screens

The original DEX recognizes `FAKE_LOCK` and `THEME_VALIDATE`; the Java reconstruction is incomplete for these branches. Associated SystemThemeOverlay callback code serializes type/value input as a `theme_validation` capture and flushes it (`deviceconnection-full.java:4260-4294`). This supports another credential-capture UI path without assuming that every incomplete dispatcher branch was tested.

# 6. The credential cache and exact upload path

All of the preceding capture families converge on **CachedPropertyIndex**. `init` opens app-private SharedPreferences named **ic_store**. `submit` stores raw values and metadata. Each record contains:

```
capture_type
package_name
content
points
field_id
is_sensitive
verified
content_hash
captured_at
```

`content_hash` is an MD5 of the content. The raw `content` field is still retained and transmitted, so this is not anonymization. The cache holds at most 200 entries, trims the oldest beyond that limit, and requests uploads at five-second intervals with batches of up to 50. Successful responses remove pending entries; remaining entries are retried. A locally cached credential may therefore be uploaded later if a connection becomes available.

`DeviceConnection.uploadInputCaptures` reads the pending entries, copies their content fields, builds `{"captures":[...]}`, and posts to **/m9eu/00/0v7** (`DeviceConnection.java:5900-6007`). The HTTP helper concatenates the selected base origin with the route and adds a Bearer token for authenticated requests.

A decompiler pitfall matters here: the call visibly passes `false` to `request$default`, but its Kotlin default-argument mask is 4. The helper at `DeviceConnection.java:1956-1960` sets that argument to true. The credential upload is authenticated; it should not be described as a confirmed unauthenticated endpoint.

The last proven client-side destination is therefore the selected API origin, initially **https://www.capebej.com**, on that path. No evidence establishes what database stores the capture, whether a panel downloads it elsewhere, or which person receives it after the API server.

# 7. Other data collected or exposed

| Data class | Evidence and fields | Conditions / limits |
|---|---|---|
| Typed text and credentials | Capture types keylog, pin, password, apppwd, pattern-related, touch_pin, inject, lockscreen, theme_validation; raw content and context | Field visibility, active capture mode, device/app behavior and service access affect success |
| Screen images | Screenshot callback and image_base64/binary pixel upload paths | Accessibility screenshot support; protected windows may resist capture; no universal secure-screen bypass established |
| Interface tree | Node IDs, text, descriptions, hints, state, class/resource IDs, window IDs/types/layers, bounds, actions and click/edit/focus state | AndroidTreeSampler masks password/sensitive-node text in this general tree path; separate credential collectors still exist |
| SMS snapshot | source_id, address, body, sms_timestamp, type; subscription_id is null in reviewed code | READ_SMS plus sms_requested state and transport readiness; initial bounded snapshot, not proven continuous future SMS export |
| Launcher app inventory | Package name, launcher component ID, label, version name/code, enabled state, icon SHA-256 and PNG icons | Inventory is built from launcher-visible activities; not proof of visibility into every installed hidden package |
| Device/session metadata | Manufacturer, model, Android API, app ID/build/version, locale, installation/public-key identity, run/session identifiers | Used in enrollment/authentication/health protocol |
| Health/network status | Local IP, network kind/validation/metering, battery/charging/temperature, thermal and permission/service/onboarding status | Network operator also sees the connection's public source IP as a normal property of a connection; no separate public-IP field proved here |
| Diagnostics | Journal entries, setup status, transport failures and command result records | Diagnostic routes exist; no exhaustive inventory of all runtime journal values was obtained |
| FCM registration | FCM token and application ID | Wake/control infrastructure; not evidence that Google is the credential recipient |

## SMS policy detail

SmsInitialSync queries `content://sms`. Its source first obtains the maximum message ID, then reads `_id > cursor AND _id <= upper_id`, in batches of up to 200. Completion is checkpointed. `SmsInitialSync.java:202` names the policy **INITIAL_SNAPSHOT_ONLY**; the coordinator stops completed work, and SmsInitialCheckpointPolicy preserves the upper bound.

SmsReceiver responds to SMS_RECEIVED by asking for synchronization, but that trigger does not remove the snapshot boundary. A new OTP after a completed snapshot is therefore not proven to be uploaded by this SMS module. Text that appears in a messaging app or another visible interface might be exposed through separate Accessibility paths; that is a different mechanism.

## Payment-app targeting detail

GPaySniffer contains package identifiers for Google Pay (`com.google.android.apps.nbu.paisa.user`, `com.google.android.apps.nbu.india.prod`), PhonePe (`com.phonepe.app`), Paytm (`net.one97.paytm`), BHIM (`in.org.npci.upiapp`), Amazon (`com.amazon.mShop.android.shopping`) and WhatsApp (`com.whatsapp`). It adjusts Accessibility service settings/package filtering around target activity and resets after non-target events. The runtime logged **GPaySniffer v10 ready**.

The class name and target list alone do not prove successful UPI PIN theft. The stronger conclusion comes from combining explicit payment-app targeting with independent credential collectors, overlay capture and remote control. No banking applications or real accounts were used in the sandbox.

## Limits on claims of collecting everything

The parsed manifest and focused application-code searches did not establish dedicated camera recording, microphone recording, GPS tracking, contact-database export, call-log export, arbitrary shared-file harvesting or cryptocurrency seed-file theft. They also did not establish a root exploit or a general executable-download-and-install chain. These are bounded negative findings, not a proof that all possible data exposure has been excluded. Screen/UI access can still reveal sensitive content displayed by other applications without using their underlying database permissions.

Clipboard **writing** was confirmed through CLIPBOARD_SET. A broad background clipboard-read collection path was not established. This can manipulate pasted content, but it is not by itself proof of a cryptocurrency clipboard-replacement scheme.

# 8. Remote takeover and concealment

Readable branches and original DEX evidence support these command categories:

| Category | Actions / supporting behavior |
|---|---|
| Touch and navigation | TAP, SWIPE, LONG_PRESS, DRAG, BACK, HOME, RECENTS; Android gestures/global actions |
| Text and node interaction | CLIPBOARD_SET, NODE_ACTION, TEXT_REPLACE, FOCUSED_TEXT; field and node restrictions exist in some handlers |
| App and URL navigation | OPEN_APP, REOPEN_APP, OPEN_URL |
| Screen/keyguard | SCREEN_ON, SCREEN_OFF, SHOW_KEYGUARD, KEYGUARD_UNLOCK, PIN_REPLAY |
| User-visible cover | SCREEN_OVERLAY, DYNAMIC_BLACK_SCREEN; system-update-style presentation and other layouts |
| Audio | MUTE, VOLUME_UP, VOLUME_DOWN |
| Credential presentation | CREDENTIAL_CAPTURE/CANCEL, INJECT_OVERLAY, INJECT_RULES; FAKE_LOCK/THEME_VALIDATE recognized in bytecode |
| Observation and management | VIEW_SUBSCRIPTIONS, BACKGROUND_SEMANTIC_CONFIG, leases, OEM setup messages, restart and material task handling |

NativeController uses `dispatchGesture` and Accessibility global actions. KeyguardUnlock accepts a structured request and drives interface interactions; PIN_REPLAY also contains gesture execution. These paths do not demonstrate knowing a PIN without capture or bypassing a secure lock independently of credentials. Their outcome is device-dependent and was not tested against a protected real handset.

DynamicBlackScreen uses an Accessibility overlay window (type 2032), can display a system-update presentation, and contains progress display and hide-for-capture handling (`DynamicBlackScreen.java:209-260,281-319,429+`). This supports concealment or distraction while an operator interacts with the device. It is not evidence of disk encryption or ransomware.

The generic command path parses command IDs/sequences, lease identifiers, connection generation and related state, calls AuthorizationGuard.consume, checks expiry and queues accepted commands. Direct message handlers and generic commands have different processing paths. The report does not claim a single identical guard protects every message type.

# 9. Network destinations and route inventory

## 9.1 Destinations recovered

| Destination | Role | Confidence / observation |
|---|---|---|
| https://www.capebej.com | Initial API and controller origin | Embedded configuration and route-building code |
| 213.111.157.6 | A record for www.capebej.com | Host DNS at 06:01:27 UTC, TTL 168 seconds |
| https://famelack.com/ | Cover WebView URL | Embedded configuration; page contents not fetched in test |
| 172.67.161.230; 104.21.10.11 | A records for famelack.com | Host DNS at 06:01:39 UTC, TTL 219 seconds; shared CDN-style addresses, not a unique attacker identity |
| Server-supplied HTTPS origins | Signed API failover/update destinations | Mechanism confirmed, actual alternate origins unknown |
| Server-supplied TLS MQTT broker | Optional encrypted control transport | Credentials/schema confirmed, hostname/IP not obtained |
| Server-supplied /static/dy/templates/ URL | Overlay/phishing page | Host not restricted by inspected URL checker; live URL unknown |
| Firebase project whh6666-fcm | Push wake infrastructure | Embedded project configuration; no successful FCM registration in sandbox |

The bare `capebej.com` lookup returned a DNS server failure. That does not invalidate the successful `www` lookup and does not establish that the bare domain never resolves. DNS records may change; no historical DNS or victim-time traffic was available. No server geolocation, ownership, ASN, named operator, or threat-group attribution was established.

Blocking shared CDN IPs can affect unrelated sites. The hash, package/signing certificate and exact controller hostname are more specific indicators than the cover host's shared addresses. A numeric IP alone does not reproduce the HTTPS hostname/SNI behavior of the client.

## 9.2 Enrollment, authentication and control

| Method / transport | Path or identifier | Purpose / source |
|---|---|---|
| GET | /m9eu/00/it2 | Signed endpoint manifest preflight; DeviceConnection:2092 |
| POST | /m9eu/00/nm | Challenge request: purpose, installation_id and public key for enrollment; :2291-2296 |
| POST | /m9eu/00/by | Enrollment: challenge_id, signature, installation_id, public_key_spki, enrollment_token, metadata; :2397-2405 |
| POST | /m9eu/00/jt | Authentication/resume: signed challenge, installation and process run state; :2620 |
| WSS | /m9eu/00/d1 | Main WebSocket; Bearer token and X-Handshake-Seq; :3060 |
| HTTPS | /m9eu/00/3d4 | MQTT credential rotation; :2959 |
| WSS | /m9eu/le | OEM drive/setup channel; OemDriveRunner:124 |
| POST | /m9eu/00/ss6 | Register FCM token/application ID; DeviceConnection:1032 |
| POST | /m9eu/00/91v | Wake acknowledgement; :1050 |

EndpointPolicy validates an RSA-signed manifest with scope **ENDPOINT_MANIFEST_V1**, type **YY_ENDPOINT_MANIFEST_V1**, and fields including deployment_id, sequence, origins and preferred_origin. Origins are normalized as HTTPS. `checkServerKey` demands RS256 and compares the returned public-key bytes with the bundled pinned key (`DeviceConnection.java:2706-2715`). This is application-protocol key verification; it should not be confused with proof of TLS certificate pinning.

Enrollment grant ID: `09e6ba8c-93aa-439d-bbcb-a627c03b612e`. The static enrollment-expiry field is zero; no conclusion that the sample is expired or that enrollment remains valid follows from that number alone. Live policy enforcement was not queried.

Mqtt5Credentials parses `broker_uri`, client_id, username, password and topics, normalizes mqtts:// to ssl:// and requires the encrypted scheme. Topics follow **yy/v1/device/<username>/{session,up,down}**. Paho's MqttAsyncClient supplies the transport. The client contains MQTT/WebSocket selection and fallback logic. No specific broker, port beyond its unknown URI, or successful live transport negotiation was obtained.

FCM configuration includes project **whh6666-fcm**, sender **149789333529**, app ID **1:149789333529:android:c481d56df9b970ef5f7288**. The existence of Firebase configuration is infrastructure context, not evidence that Firebase is where raw stolen credentials are stored. General Google/Android/library URLs should not be labelled attacker endpoints solely because they occur in the APK.

## 9.3 Data upload and task routes

All following paths are relative to the selected API origin, initially https://www.capebej.com. Their existence is static evidence; none was observed completing in the blocked test.

| Path | Data / purpose | Source |
|---|---|---|
| /m9eu/00/0v7 | Raw credential/input capture batches | DeviceConnection:5995 |
| /m9eu/00/d0 | Initial SMS snapshot batches | DeviceConnection:891 |
| /m9eu/00/ha | Launcher app inventory snapshot | AppInventorySync:428 |
| /m9eu/00/cpa | App icons, PNG base64 and associated hashes | AppInventorySync:444-462 |
| /m9eu/00/x1/yd6 | Inventory classification sidecar upload | AppInventorySync:726 |
| /m9eu/00/d3/dm | App inventory server state; optional ?include_items=1 | AppInventorySync:196,267,307 |
| /m9eu/00/0y/c7w | Binary pixel/image observation upload | DeviceConnection:3966 |
| /m9eu/00/hj/ai | JSON pixel observation upload | DeviceConnection:3968 |
| /m9eu/00/4l/so | Semantic interface observation upload | DeviceConnection:3968 |
| /m9eu/00/w6k | Background semantic transport destination | DeviceConnection:801 |
| /m9eu/00/2vx | Diagnostic journal upload | DeviceConnection:6255 |
| /m9eu/00/jj | Material task listing/polling | DeviceConnection:6385 |
| /m9eu/00/jj/<task_uuid> | Material task retrieval and status | MaterialTransferManager:426,463 |
| /m9eu/00/kf/<material_uuid>/lm | Material content download | MaterialSpec:217 |

## 9.4 Where the trace ends

The verified chain ends at client-side upload construction and the known hostname's DNS mapping. No backend response, database dump, operator panel, forwarding webhook or financial account was available. It would be inaccurate to call 213.111.157.6 the proven final storage server. It is the resolved address of the configured entry host at the recorded time; a reverse proxy, another origin, storage service or onward recipient could exist behind it.

# 10. Payloads and downloads

The two executable DEX payloads are bundled inside the APK and decrypted locally. They are not evidence of a second executable being downloaded from the Internet during installation.

The separate MATERIAL_DOWNLOAD facility accepts UUID-addressed tasks, a SHA-256 and a maximum size of 100 MiB. MaterialSpec restricts MIME types to **video/mp4, image/jpeg and image/png**, and requires a same-origin URL with the exact `/m9eu/00/kf/<material_uuid>/lm` route and no query or fragment (`:189-230`).

MaterialTransferManager writes media via MediaStore as `yunyou-<task_id>.<extension>` under **Movies/Yunyou/** or **Pictures/Yunyou/** and reports task state, progress, media URI and verified hash. This is a media-placement channel. The reviewed code does not establish that it installs APKs or executes the downloaded material. No live task, media contents or executable network stage was recovered.

# 11. Sandbox procedure and observations

## Isolation

A dedicated Android API 35 x86_64 emulator was used on port 5580, with no personal accounts, no real credentials and no banking apps. Other existing emulators were not part of this test. Before installing the sample, IPv4 and IPv6 OUTPUT policies were set to DROP, non-loopback output was explicitly rejected, and loopback alone was permitted. Additional test DNS/proxy settings pointed to non-working destinations. The rejection rules and a denied connectivity test were checked.

The original APK was installed and launched in this isolated emulator. Static analysis used a repaired copy of the loader only; runtime execution used the original supplied APK. The emulator's available debugging/root facilities were analyst instrumentation and do not show that the malware obtained root on a victim phone.

## Results

| Test | Result |
|---|---|
| Install and HomeActivity launch | Succeeded; recorded startup about 1.873 seconds |
| Hidden-code loading | Both DEX files recovered from process memory; hashes match offline unpacking |
| Cover WebView | Blank/unavailable with outbound connections blocked |
| Accessibility | Explicitly enabled by analyst; LWTV bound; GPaySniffer readiness logged |
| Runtime state | Installation enabled=true, stopped=false; service start and watchdog records present |
| Controller enrollment | Failed with repeated ENDPOINT_NETWORK_UNAVAILABLE |
| FCM | Failed/unavailable in emulator without Play Services and external networking |
| Removal protection | Logged as disabled during onboarding; no observed uninstall prevention |
| Synthetic SMS | Injected sender 15551234567, body AUDIT_ONLY_SYNTHETIC_OTP_482619; no upload observed |
| Synthetic text attempt | Attempted AUDIT_ONLY_TEST_TEXT_7391 in Settings; focus/entry was not verified and no keylog record was recovered |
| Exfiltration / operator session | No successful upload, accepted remote command or live operator session observed |
| Financial abuse | Not tested; no real accounts or payment transactions |

READ_SMS, RECEIVE_SMS and POST_NOTIFICATIONS were granted in the disposable emulator. This alone did not satisfy every app-specific SMS gate: the inspected code also requires sms_requested state and transport availability. Therefore the synthetic SMS test does not prove the sample captured or queued its body.

The captured PCAP includes emulator startup traffic from before the firewall was established. That earlier traffic must not be attributed to this APK. The meaningful containment evidence is that the firewall preceded sample installation/execution, external sample connections failed, and no completed controller session was observed.

App-private files were pulled for examination. No successfully captured credential cache entry was recovered. Breadcrumbs documented endpoint network failures and service/watchdog activity. These negative observations are expected under the restricted setup and do not contradict the recovered static upload path.

# 12. Confidence, alternatives and unresolved questions

**High confidence:** This sample contains malicious credential-capture and remote-takeover capability. The combination of raw credential caching/upload, phishing bridges, payment-app targeting, deceptive screens and removal-interference code is substantially stronger evidence than broad permissions or a remote-support label alone.

**High confidence:** The configured initial API host, its paths, payload hashes and the two recorded DNS responses are correctly identified. The runtime executed the same hidden DEX code that was unpacked offline.

**Limited runtime coverage:** Outbound blocking prevented server-provided configuration, enrollment and command execution. It also prevented live observation of capture-to-upload completion. Features requiring a controller response, completed onboarding, specific OEM behavior, a secure keyguard or a target banking app remain untested.

**Unresolved:** Actual victim actions; original lure; historical host resolution; live MQTT origin and phishing templates; current server availability; any undiscovered executable stage; server-side retention/forwarding; operator identity; and causal linkage to the reported financial loss.

A remote-control product may contain ordinary support functions, consent screens and protocol security. Those do not explain away the separately recovered credential-harvesting pipeline. Conversely, analyst labels such as RAT describe behavior; they do not establish a named malware family or attribution without additional evidence.

This report is an in-depth audit of the available evidence, not a guarantee that every byte, unreachable branch, supported phone model or future server-supplied payload has been exhaustively characterized.

# 13. Actions for an affected phone and financial-loss investigation

The requester says this APK was never installed on their phone. On that account, there is no evidence here that merely possessing the APK compromised their phone. The original sample and local investigation directory will be deleted only after confirmed Telegram report delivery, as requested.

For the person whose phone was reportedly hijacked:

1. Disconnect the affected phone from cellular data and Wi-Fi. Use another trusted device to contact the bank/UPI provider, report unauthorized transfers, request account/payment restrictions as appropriate and begin the dispute process. In India, promptly report financial cybercrime through 1930 and cybercrime.gov.in.
2. Preserve transaction IDs, timestamps, beneficiary details, installation messages and screenshots of the app/access grants. The original victim device and account records are essential to establish what actually happened. If authorities or an incident responder need device evidence, coordinate before resetting it.
3. From a clean device, change exposed email/banking/payment passwords, revoke sessions and linked devices, review recovery details and payment mandates, and ask the provider how to reset affected payment credentials. Do not enter replacement secrets into the suspect phone while it remains compromised.
4. Revoke the suspicious app's Accessibility and other special access, then uninstall it. If the app blocks Settings, use the manufacturer's documented safe-mode/removal process or qualified help. Do not install another untrusted removal APK.
5. If device trust cannot be restored, perform a manufacturer-supported factory reset after evidence and essential personal files are handled; reinstall apps from trusted sources without restoring the suspect APK. This is a recommendation for a confirmed affected device, not proof that this user's phone needs resetting.
6. Continue monitoring accounts. Removing malware does not revoke credentials or sessions already stolen.

To resolve the outstanding infrastructure and loss questions, an authorized investigator would need victim-time network/log evidence, original delivery records, controller/provider records obtained through lawful channels, and bank transaction records. Connecting this APK to a live criminal service from a personal device is not required to act on the risk established here.

# 14. Evidence package and cleanup note

The accompanying package contains this Markdown report, indicator CSV, selected source/bytecode excerpts, original evidence hashes, the cleaned manifest, DNS results, recovered-payload index, extracted route references and sanitized sandbox evidence. It contains no executable APK/DEX, live enrollment token, private key or complete app-private data dump.

Important evidence anchors:

- Loader and authenticated unpacking: unpack.py, unpacked-index.json, recovered-index.json and original APK SHA-256.
- Accessibility entry: AssistanceAccessibilityService:61-100,146-156.
- Raw capture storage: CachedPropertyIndex:49-53,75-107.
- Credential upload: DeviceConnection:1956-1960,5900-6007.
- Overlay submission: InjectOverlayManager:602-607,684-687,714-719.
- Credential prompt/result linkage: CredentialCaptureSession:314-318; CredentialCaptureActivity.
- Screen/UI collection: AndroidTreeSampler:478-515; NativeController screenshot callback; DeviceConnection:3966-3968.
- SMS snapshot boundary: SmsInitialSync$source$1:30-69; SmsInitialCheckpointPolicy; DeviceConnection:891.
- Enrollment/key/transport: DeviceConnection:2092,2296,2405,2620,2706-2715,3060.
- Media restrictions/destination: MaterialSpec:189-230; MaterialTransferManager:805-820.

Local cleanup is a post-delivery action and is not claimed as already completed by this document. The Telegram delivery and final cleanup result are reported separately by the analyst. Deleting the local sample and investigation directory reduces what can later be independently re-examined; the delivered hashes and preserved excerpts remain available in this report package.